

Procura Local e Otimização

Subida da encosta e Recozimento simulado

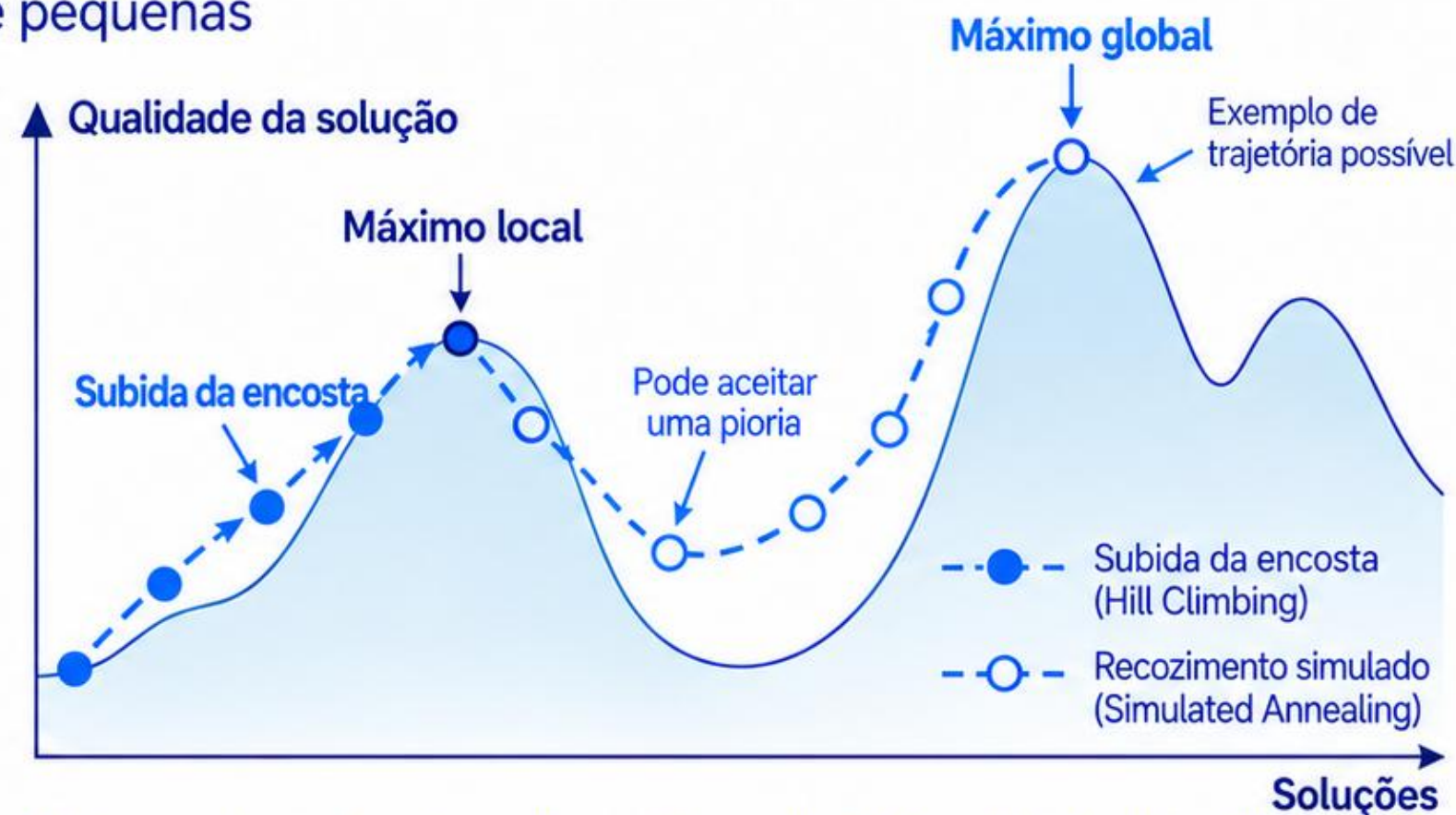
Partir de uma solução candidata e melhorá-la através de pequenas alterações. Cada alteração produz uma solução vizinha.

● Subida da encosta (Hill Climbing)

Passa para uma solução vizinha melhor.
Pode parar num ótimo local, sem encontrar a melhor solução possível.

○ Recozimento simulado (Simulated Annealing)

Pode aceitar temporariamente soluções piores para escapar de ótimos locais. Essa possibilidade diminui ao longo da execução.



Neste exemplo, quanto maior a altura, melhor a solução. Um máximo local é melhor do que os seus vizinhos; o máximo global é o melhor de todo o espaço de procura.



Exemplo — 8 rainhas

Cada configuração do tabuleiro é uma solução candidata. Mover uma rainha gera uma solução vizinha.
O objetivo é **minimizar** o número de pares de rainhas que se atacam, até atingir **zero**.

Procura de caminhos e procura local

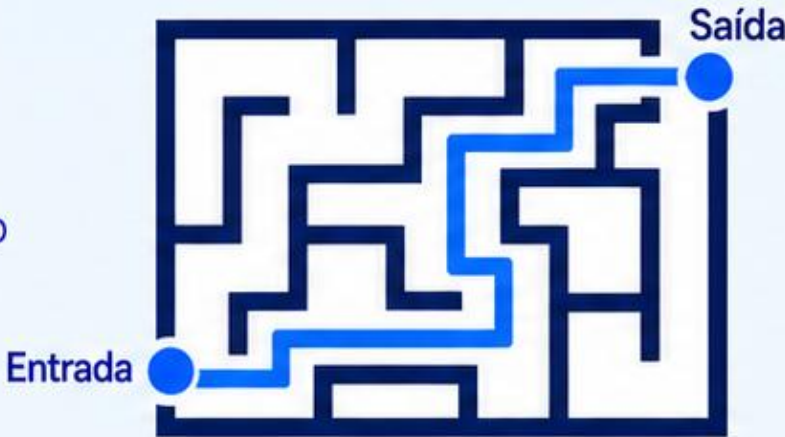
O que interessa como resultado: a sequência de ações ou a configuração encontrada?

Procura de caminhos

Procura uma sequência de ações que liga o estado inicial a um estado objetivo.

Exemplo: num labirinto, é necessário indicar os movimentos desde a entrada até à saída.

O custo do caminho é a soma dos custos dos movimentos.



Interessa a sequência de ações.

Exemplos: labirintos e percursos entre cidades.

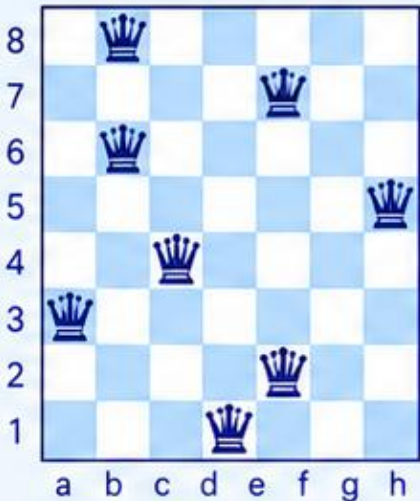
Procura local

Parte de uma configuração candidata e gera configurações vizinhas através de pequenas alterações. Uma função de avaliação compara a sua qualidade.

Nas 8 rainhas, altera-se a posição das peças para reduzir os conflitos.

Interessa a configuração encontrada.

Exemplos: horários e problema das 8 rainhas.



Configuração candidata

Como avaliar?

Contar os pares de rainhas que se atacam.

Menos conflitos = melhor configuração.

Zero conflitos = solução válida.

Aspeto	Procura de caminhos	Procura local
Resultado	Sequência de ações até ao objetivo	Configuração com boa qualidade
Como avança	Explora caminhos possíveis	Modifica a configuração atual
Memória	Depende do algoritmo; pode guardar muitos estados	Nos métodos estudados, centra-se na configuração atual

- **Métodos locais:** Subida da encosta (*Hill Climbing*) e Recozimento simulado (*Simulated Annealing*).
- A procura local pode terminar sem encontrar a melhor solução possível.

O que é um problema de otimização?

Na procura local, modificamos uma configuração para a melhorar. Para isso, precisamos de definir o que significa «melhor».

Otimizar é procurar a melhor solução válida, segundo um critério definido.

Conceito	Significado	Exemplo: 8 rainhas
Solução candidata	Uma configuração a avaliar; pode ainda violar restrições.	Uma disposição de oito rainhas num tabuleiro de 8 × 8.
Alteração local	Uma pequena mudança que gera uma configuração vizinha.	Mover uma rainha para outra posição.
Função objetivo	Mede a qualidade para comparar configurações.	Contar os pares de rainhas que se atacam. Pretende-se minimizar esse número.
Restrições	Condições que a solução final tem de cumprir.	Colocar oito rainhas sem partilharem linhas, colunas ou diagonais.

Solução válida

Tem zero pares de rainhas em conflito.



Solução ótima neste exemplo

Zero é o menor número de conflitos possível. Por isso, uma solução válida também é ótima para esta função objetivo.



Menos conflitos = melhor configuração.

Melhorar não significa resolver: enquanto houver conflitos, a configuração ainda não é uma solução válida.

Estados, movimentos e vizinhança

Já sabemos avaliar uma configuração pelo número de conflitos. Agora precisamos de a representar e definir como a podemos alterar.

1. Representar o estado

Estado $s = [2, 1, 5, 4, 4, 3, 2, 2]$

	1	2	3	4	5	6	7	8
8								
7								
6								
5			♔					
4				♔	♔			
3						♔		
2	♔						♔	♔
1		♔						

Uma rainha por coluna.
A posição na lista identifica a coluna; o número indica a linha.

Coluna	1	2	3	4	5	6	7	8
Linha	2	1	5	4	4	3	2	2

O primeiro número, 2, coloca a rainha da coluna 1 na linha 2.

Números repetidos indicam rainhas na mesma linha, portanto em conflito.

Evita conflitos de coluna, mas pode ter conflitos de linha e de diagonal.

2. Mover uma rainha

Mover uma rainha para outra linha, mantendo-a na mesma coluna.

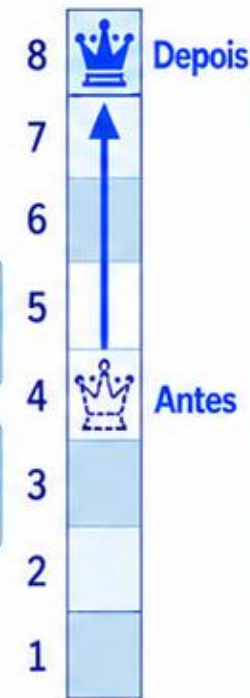
Exemplo: na coluna 5, passar da linha 4 para a linha 8.

Antes: $[2, 1, 5, 4, 4, 3, 2, 2]$

Depois: $[2, 1, 5, 4, 8, 3, 2, 2]$

Só o quinto número muda.

Coluna 5



3. Gerar vizinhos

A **vizinhança** reúne todas as configurações obtidas com um único movimento.

8
rainhas à escolha

7
outras linhas para cada rainha

$8 \times 7 = 56$ vizinhos

Ser vizinho não significa ser melhor.

O movimento pode diminuir, manter ou aumentar o número de conflitos.

Quantas configurações existem?

Cada uma das 8 colunas tem 8 posições possíveis para a sua rainha.

$8^8 = 16\,777\,216$ configurações

Muitas destas configurações contêm conflitos. Só zero conflitos corresponde a uma solução válida.

Como avaliar uma configuração das 8 rainhas?

Já sabemos representar um estado e gerar vizinhos. Agora contamos os conflitos para comparar configurações.

Quando existe conflito?

Com uma rainha por coluna, basta verificar linhas e diagonais.

Mesma linha

$$r_i = r_j$$

As duas rainhas têm o mesmo número de linha.

Mesma diagonal

$$|r_i - r_j| = |i - j|$$

A distância entre linhas é igual à distância entre colunas.

i e j identificam as colunas; r_i e r_j indicam as linhas. As barras $| |$ representam o valor absoluto.

Exemplos no estado anterior

$$s = [2, 1, 5, 4, 4, 3, 2, 2]$$



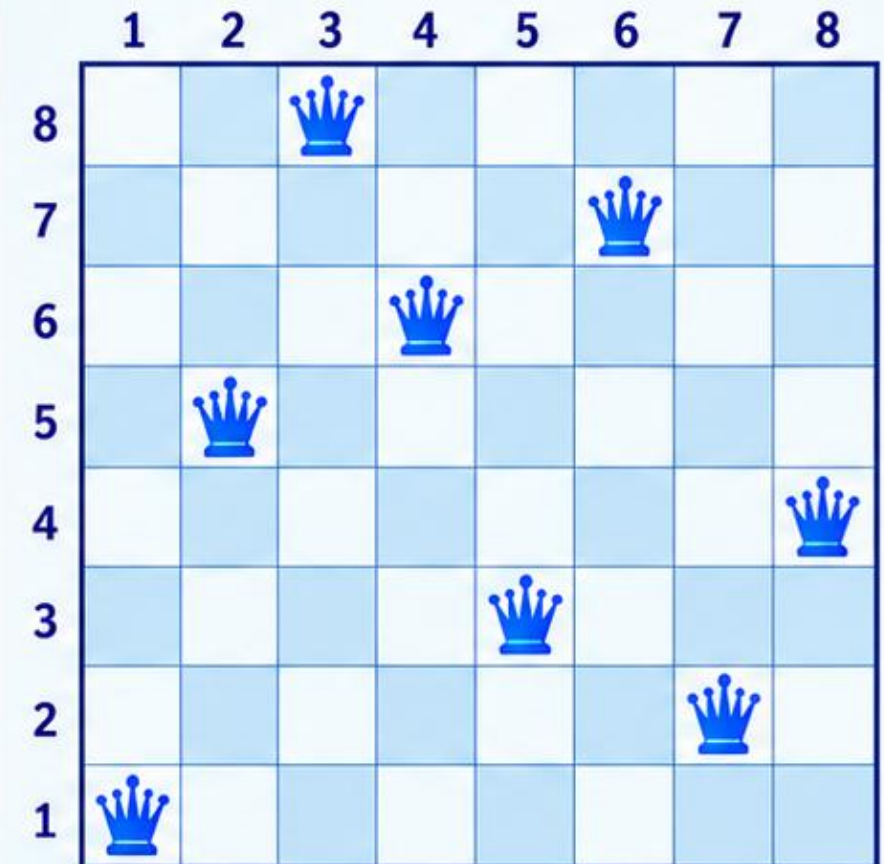
Colunas 4 e 5:
mesma linha.

Colunas 1 e 2:
mesma diagonal.

Colunas 1 e 3:
sem conflito.

São destacados dois
exemplos de conflito.
No total, $h(s) = 10$.

Exemplo sem conflitos



$$s = [1, 5, 8, 6, 3, 7, 2, 4]$$

$h(s) = 0$: solução válida

Função de avaliação

$h(s)$ = número de pares
de rainhas em conflito.

Cada par é contado
apenas uma vez.

$8 \times 7 \div 2 = 28$
pares a verificar.

Quanto menor $h(s)$, melhor.
Com zero conflitos, encontramos
uma solução válida.

A paisagem de procura nas 8 rainhas

Minimização: menos conflitos é melhor

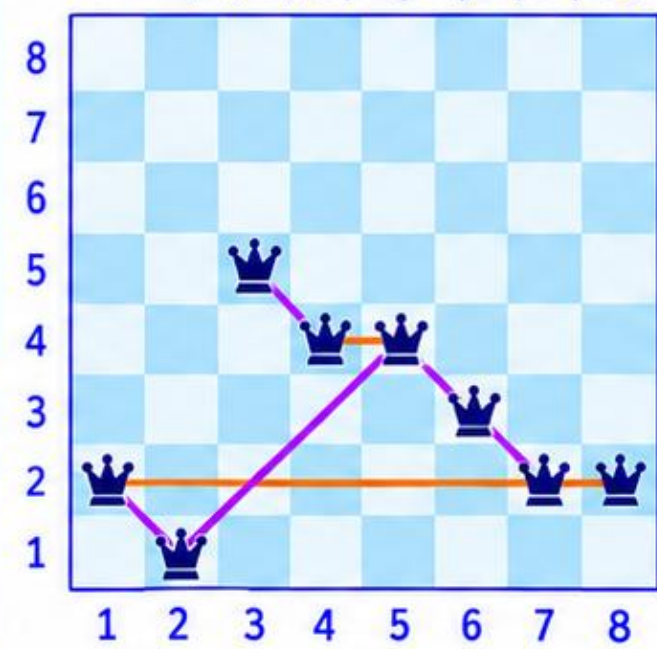
Como contamos os conflitos?

$h(s)$ conta os pares de rainhas na mesma linha ou diagonal, mesmo que exista outra rainha pelo meio. **Cada par conta uma única vez.**
É uma medida de alinhamento; não conta apenas ataques diretos de xadrez.

Exemplo no tabuleiro A:

Na linha 2: rainhas nas colunas 1, 7 e 8.
Três rainhas alinhadas formam três pares: (1,7), (1,8) e (7,8).
O par (1,8) conta, apesar da rainha na coluna 7.

A: $h(s) = 10$
 $s = [2, 1, 5, 4, 4, 3, 2, 2]$



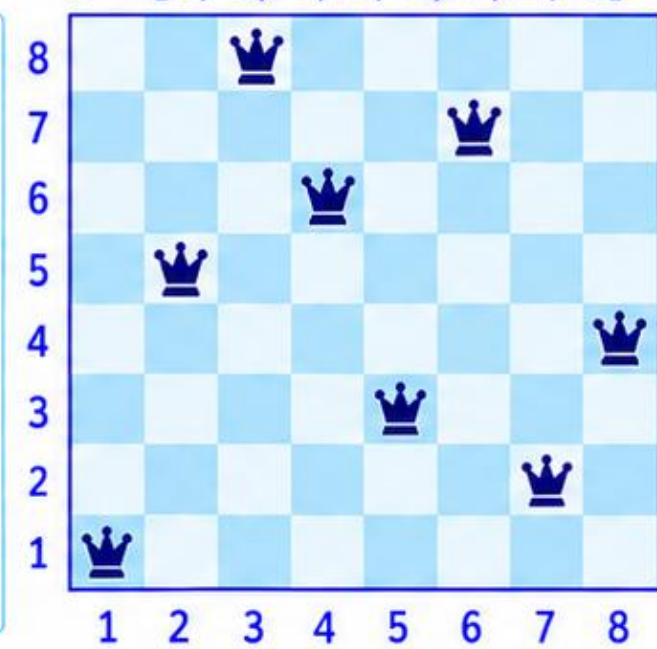
10 pares em conflito
Pares identificados pelas colunas

Linha: 4 pares
(1,7), (1,8), (4,5), (7,8)

Diagonal: 6 pares
(1,2), (2,5), (3,4), (5,6), (5,7), (6,7)

$h(s) = 4 + 6 = 10$
Algumas ligações sobrepõem-se.

B: $h(s) = 0$
 $s = [1, 5, 8, 6, 3, 7, 2, 4]$



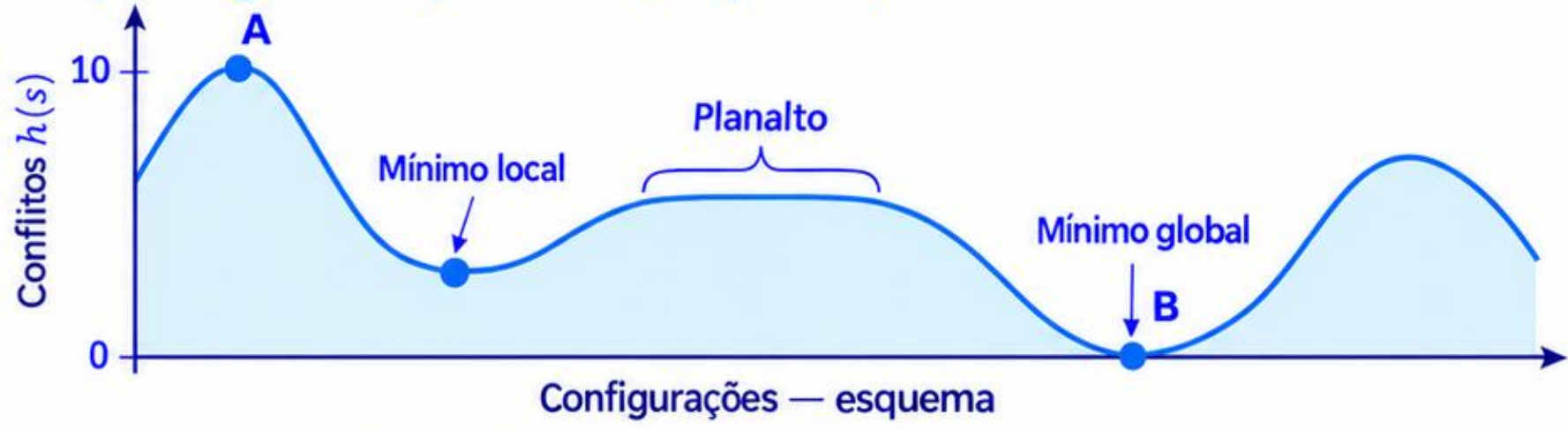
Como interpretar a paisagem?

Um ponto:	uma configuração.
Altura:	número de conflitos $h(s)$.
Movimento:	mover uma rainha para outra linha da mesma coluna.
Descer:	passar para um vizinho com menos conflitos.

Definições

Mínimo local:	nenhum dos 56 vizinhos tem menos conflitos; pode ainda haver conflitos.
Planalto (plateau)	configurações vizinhas com o mesmo número de conflitos.
Mínimo global:	$h(s) = 0$; nenhuma dupla partilha linha ou diagonal.

A paisagem de procura (esquema)



Um método que só aceita melhorias pode parar num mínimo local com conflitos.

Representação esquemática: os estados são discretos. A distância no desenho não define a vizinhança. A e B não são vizinhos.

Como funciona a procura local

Partimos de uma configuração, avaliamos vizinhos e decidimos se aceitamos uma alteração. Repetimos até cumprir um critério de paragem.



Um movimento: coluna 5, da linha 4 para a linha 8

ANTES
Configuração
[2, 1, 5, 4, 4, 3, 2, 2]
 $h(s) = 10$
Coordenadas
(coluna, linha):
(1, 2), (2, 1), (3, 5),
(4, 4), (5, 4), (6, 3),
(7, 2), (8, 2).

Mover apenas uma rainha

DEPOIS
Configuração
[2, 1, 5, 4, 8, 3, 2, 2]
 $h(s) = 6$
Coordenadas
(coluna, linha):
(1, 2), (2, 1), (3, 5),
(4, 4), (5, 8), (6, 3),
(7, 2), (8, 2).

Laranja: mesma linha

Violeta: mesma diagonal

Movimentos da rainha da coluna 5

Nova linha	Conflitos
1	7
2	10
3	9
5	9
6	7
7	7
8	6

A tabela mostra 7 dos 56 vizinhos.
O movimento para a linha 8 é o melhor entre os 56 vizinhos deste estado.

De 10 para 6 conflitos:
melhorámos, mas ainda não resolvemos.

Como contamos conflitos
Contamos pares na mesma linha ou diagonal, mesmo com outra rainha pelo meio.

CrITÉRIOS de paragem

- Parar: zero conflitos; limite de tempo ou avaliações; critério do algoritmo.
- Na versão que só aceita melhorias, parar se nenhum vizinho for melhor.

Subida da encosta (Hill Climbing)

Escolher o melhor vizinho e avançar apenas se houver melhoria.

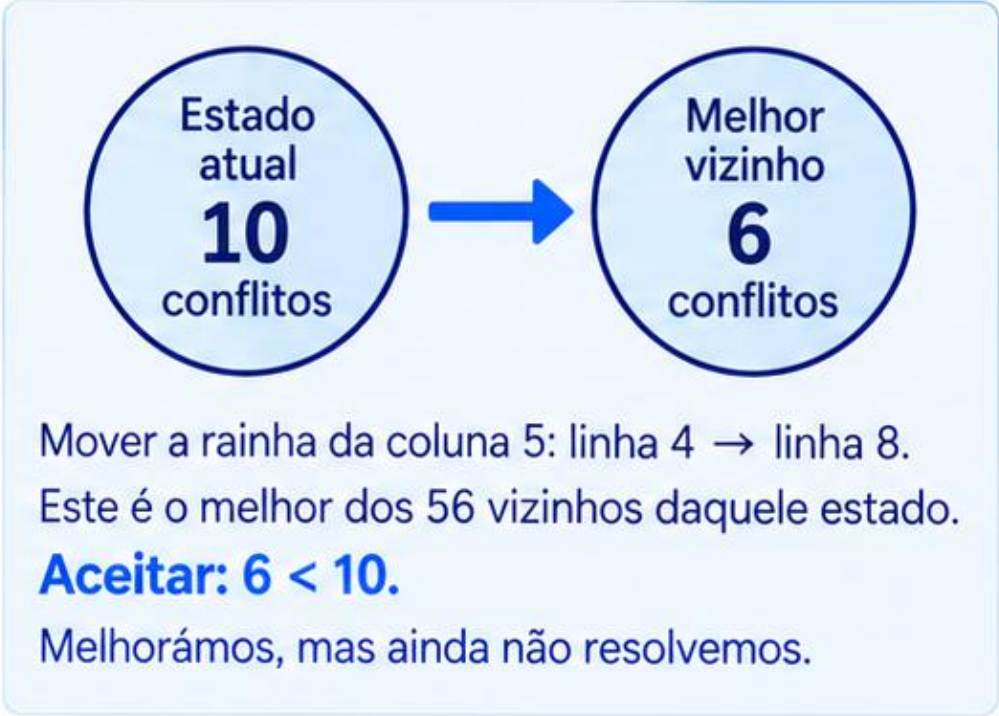
Versão de melhor melhoria • Exemplo das 8 rainhas: minimizar conflitos

1. Como decide

- 1 Avaliar os 56 vizinhos do estado atual.
- 2 Escolher o vizinho com menos conflitos.
- 3 Avançar apenas se reduzir os conflitos.
- 4 Repetir; parar com zero conflitos ou sem melhoria.

Guloso: escolhe a melhor melhoria imediata, sem antecipar os movimentos seguintes.

2. Do exemplo anterior



3. Regra de aceitação

$$h(v) < h(s)$$

s : estado atual
 v : melhor vizinho
 h : número de pares em conflito
Se vários vizinhos tiverem o mesmo menor valor, escolher um deles.

Porque se chama «subida»?
Subir significa melhorar a qualidade. Nas 8 rainhas, isso corresponde a diminuir os conflitos.

4. Quando avança e quando para

Estado atual	Melhor vizinho	Decisão
10 conflitos	6 conflitos	Avançar: existe melhoria
6 conflitos	6 conflitos	Parar: não existe melhoria estrita
6 conflitos	7 conflitos	Parar: todos os vizinhos são piores

As duas últimas linhas são situações hipotéticas. Nesta versão, não se aceitam empates nem piorias.

Parar sem conseguir melhorar não significa ter resolvido o problema.

Hill Climbing: regra de decisão

Procura local que avança para um vizinho melhor até não ser possível melhorar.

O Hill Climbing é um algoritmo de procura local que, a partir de um estado inicial, move-se iterativamente para um vizinho com melhor valor da função objetivo (qualidade), parando quando não existe nenhum vizinho que melhore o estado atual.

Algoritmo (pseudocódigo)

```
1  atual ← estado inicial
2  repetir:
3      se N(atual) estiver vazio, devolver atual
4      próximo ← melhor vizinho de atual
5      se próximo não melhora atual:
6          devolver atual
7      atual ← próximo
```

Função objetivo

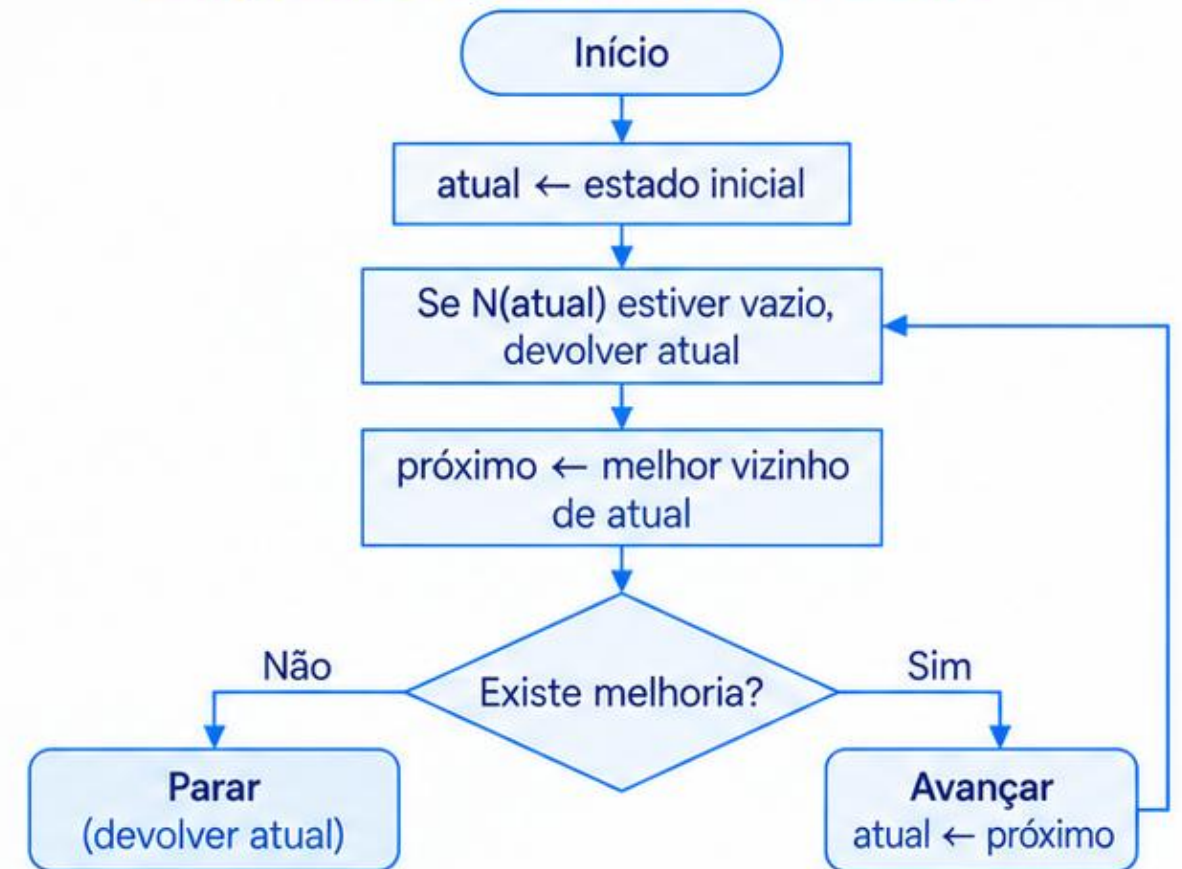
Maximizar: aumentar a qualidade

s' é melhor que s se
 $f(s') > f(s)$

Minimizar: reduzir o custo

s' é melhor que s se
 $f(s') < f(s)$

Diagrama de fluxo da decisão



Notas importantes

- 1 Uma iteração de melhor melhoria avalia até $|N(s)|$ vizinhos.
- 2 Nas 8 rainhas, são 56 candidatos.
- 3 Sem movimentos laterais, cada movimento aceite melhora estritamente a função.
- 4 Em espaço finito, termina; não garante ótimo global.

Hill Climbing nas 8 rainhas

Procura local iterativa que, em cada passo, escolhe um vizinho com menor número de conflitos.

1. Problema e função de avaliação

Colocar 8 rainhas num tabuleiro 8×8, uma por coluna, sem que se ataquem. Um estado é representado por um vetor de filas $r = [r_1, r_2, \dots, r_8]$, onde r_j é a fila da rainha na coluna j (1 a 8).

Função de avaliação (heurística):

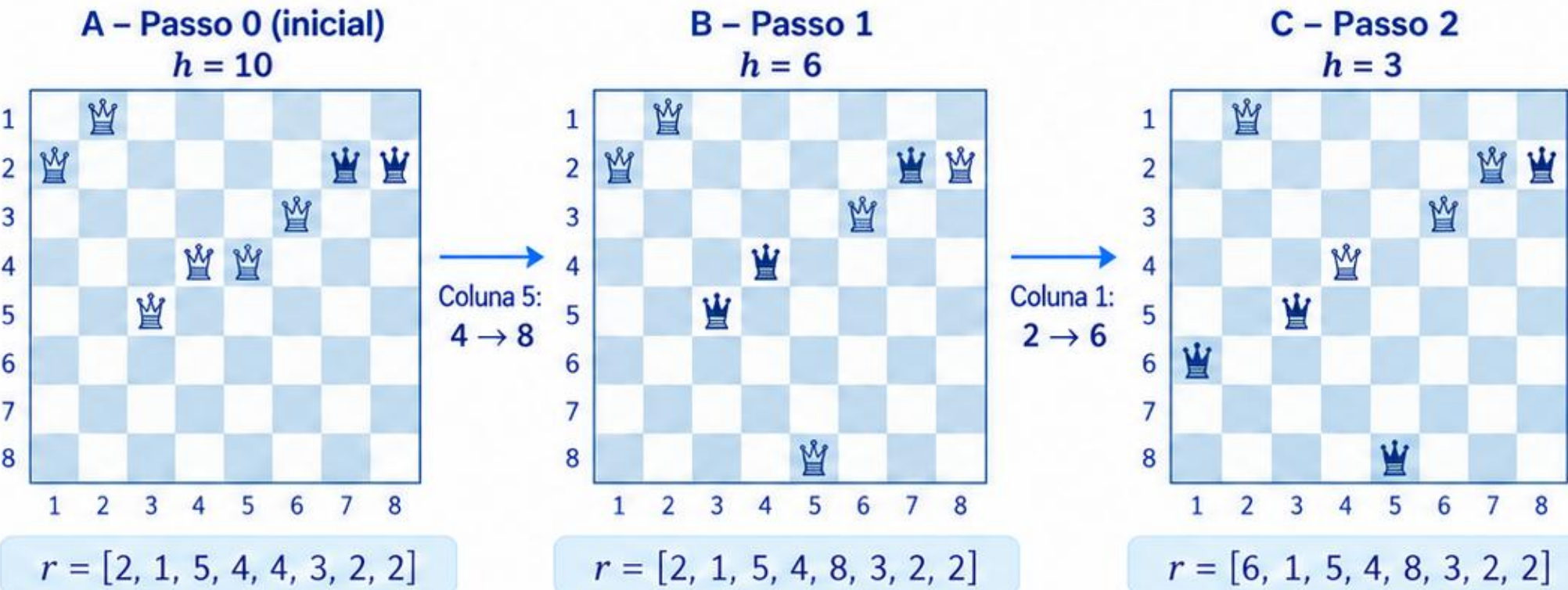
$$h(r) = \sum_{1 \leq i < j \leq 8} I(r_i = r_j \vee |r_i - r_j| = |i - j|)$$

$h(r)$ é o número de pares de rainhas em conflito (mesma fila ou na mesma diagonal). O objetivo é minimizar $h(r)$; solução quando $h(r) = 0$.

2. Hill Climbing (algoritmo)

- 1 Gerar o estado inicial r .
- 2 Avaliar todos os vizinhos (mover a rainha de cada coluna para outra fila).
- 3 Escolher o vizinho com menor $h(r)$ (desempatar de forma consistente, ex.: menor coluna e depois menor fila).
- 4 Se $h(r') < h(r)$, mover para r' e repetir.
- 5 Se nenhum vizinho tiver h menor, parar (mínimo local).

3. Traço de execução (três estados)



4. Resumo dos passos

Passo	Vetor de filas r	$h(r)$	Movimento
0	$[2, 1, 5, 4, 4, 3, 2, 2]$	10	Estado inicial
1	$[2, 1, 5, 4, 8, 3, 2, 2]$	6	Coluna 5: 4 → 8 ($\Delta h = -4$)
2	$[6, 1, 5, 4, 8, 3, 2, 2]$	3	Coluna 1: 2 → 6 ($\Delta h = -3$)

5. Observações

- Melhorias: $\Delta h = -4$ e -3 .
- Avaliar todos os movimentos e desempatar de forma consistente.
- $h = 3$ ainda não é solução; a execução continua se existir melhoria.
- Hill Climbing pode ficar preso em mínimos locais com $h > 0$.

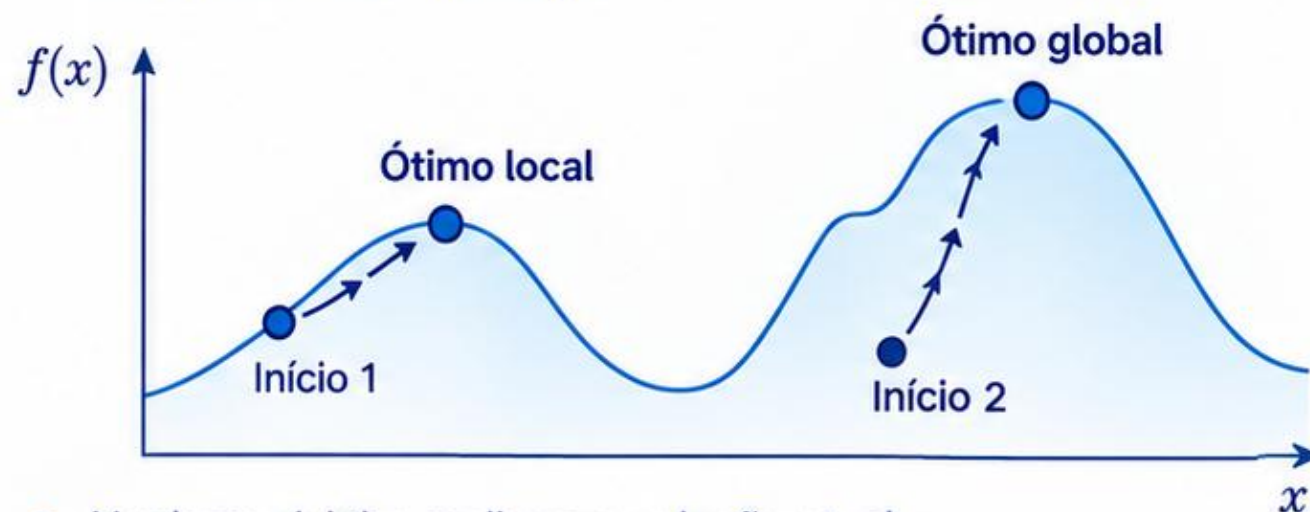
Cada passo escolhe um vizinho com menor número de conflitos.

Ótimos locais

Conceitos e definições

Um **ótimo local** é uma solução para a qual nenhum vizinho na vizinhança definida tem um valor da função objetivo melhor do que o da solução atual.

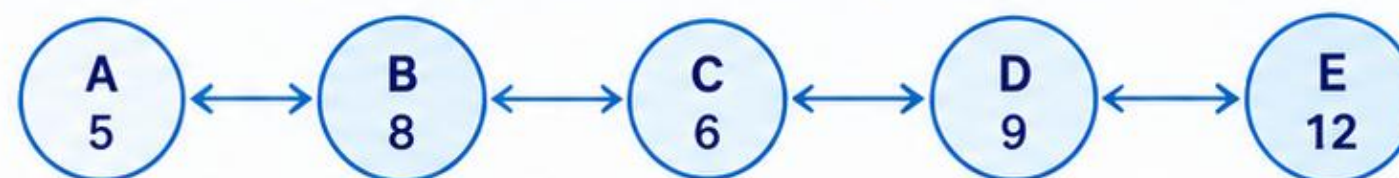
- Pode existir uma solução melhor noutra região do espaço de busca (**ótimo global**).
- O ponto de partida e o algoritmo de busca influenciam o resultado obtido.
- Ótimos são sempre definidos relativamente ao espaço de busca e à vizinhança.



- Nenhum vizinho melhora a solução atual.
- Pode existir uma solução melhor noutra região.
- O ponto de partida influencia o resultado.

Exemplo concreto: busca de subida de encosta (HC)

Considere um espaço de busca discreto com cinco estados em cadeia (apenas vizinhos adjacentes) e os seguintes valores da função objetivo:



Execução da busca de subida de encosta

- 1 Início em A (valor 5). Vizinhos: apenas B (8). Como $8 > 5$, move para B.
- 2 Em B (valor 8). Vizinhos: A (5) e C (6). Nenhum vizinho tem valor superior a 8, logo o algoritmo pára em B (**ótimo local**).
- 3 Para alcançar E (valor 12), seria necessário atravessar C, com valor 6, que é pior do que o valor atual 8, o que não é permitido pela HC.
- 4 Assim, o algoritmo devolve B, embora E seja a melhor solução do espaço (**ótimo global**).

Algoritmo (busca de subida de encosta)

```
s ← estado inicial
repita
  s* ← arg maxv ∈ N(s) f(v)
  se f(s*) > f(s) então s ← s*
  caso contrário parar
até paragem
devolver s
```

Critério de ótimo local

$$f(s) \geq f(v) \quad \forall v \in N(s)$$

onde $N(s)$ é o conjunto de vizinhos de s no espaço de busca.

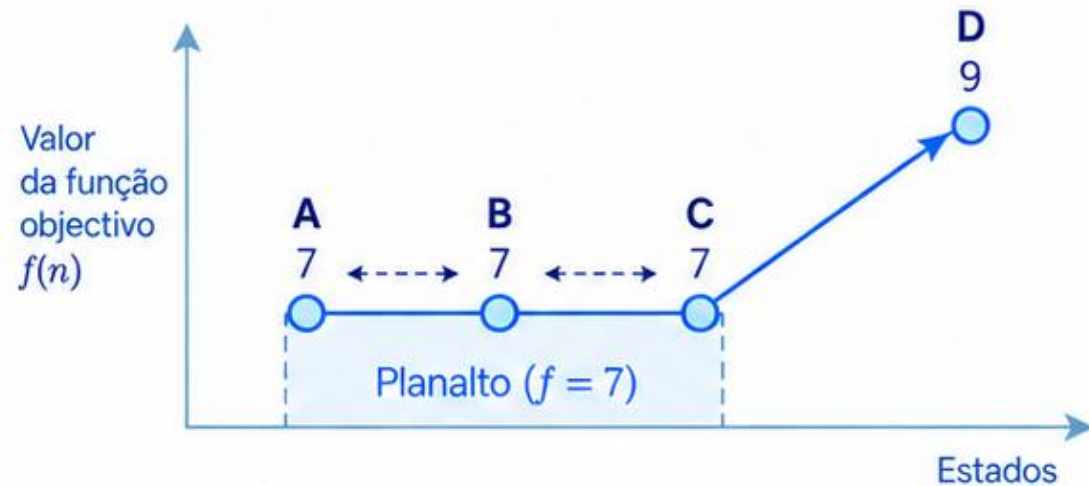
Nas 8 rainhas, pode haver estagnação com $h > 0$.
Ótimo é sempre definido relativamente ao espaço e à vizinhança.

Plateaus: regiões sem melhoria

Um plateau (planalto) é uma região do espaço de estados em que vários estados têm o mesmo valor da função objectivo. Nestas regiões não existe melhoria imediata, o que pode fazer o **Hill Climbing** estrito parar, mesmo quando existe uma solução melhor alcançável.

1. Planalto com saída

Vários estados têm o mesmo valor, mas existe um caminho para um estado melhor, alcançável através de movimentos laterais (para estados de igual valor).



Exemplo (cadeia discreta)

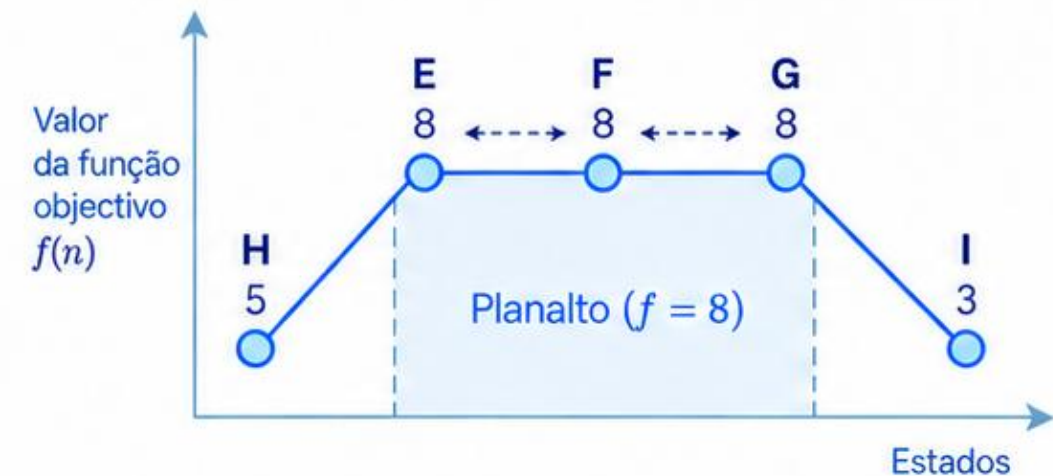
Estados e valores da função objectivo:

Estado	A	B	C	D
Valor $f(n)$	7	7	7	9
Vizinhos	B	A, C	B, D	C

- 1 A partir de A, a versão estrita não avança, porque $f(B) = 7 = f(A)$ (não é melhoria).
- 2 Movimentos laterais permitem atravessar B e C até D, onde $f(D) = 9$.
- 3 Num patamar sem saída, movimentos laterais podem causar ciclos (ex.: $A \leftrightarrow B$).

2. Máximo local plano

Vários estados têm o mesmo valor e todos os estados vizinhos fora do patamar têm valor inferior. Não existe uma saída para um estado melhor.



3. Hill Climbing e plateaus

Na versão estrita, o algoritmo apenas aceita estados com melhoria estrita:

$$n_{t+1} = \begin{cases} n' & \text{se } f(n') > f(n_t) \\ n_t & \text{caso contrário} \end{cases}$$

Se nenhum vizinho tem valor superior ao atual, o algoritmo para.

Com movimentos laterais limitados, aceita-se também $f(n') = f(n_t)$ por um número máximo de passos consecutivos.

Movimentos laterais limitados podem ajudar

- Permitem atravessar planaltos com saída e alcançar estados melhores.
- Podem causar ciclos em planaltos sem saída.
- **Regra prática:** limitar movimentos laterais consecutivos ($L_{\max}$) e reiniciar ao atingir o limite.

Cristas e vizinhança

O papel da vizinhança em problemas com cristas

Definição e questão

Uma **crista** é uma zona alongada de bons valores da função objetivo, onde o máximo (ou mínimo) se estende numa direção específica. Algoritmos de busca local movem-se apenas para soluções na vizinhança da solução atual. Se a vizinhança for restrita, o algoritmo pode progressar lentamente ou ficar bloqueado numa crista oblíqua, mesmo quando existe uma direção favorável de subida/descida.

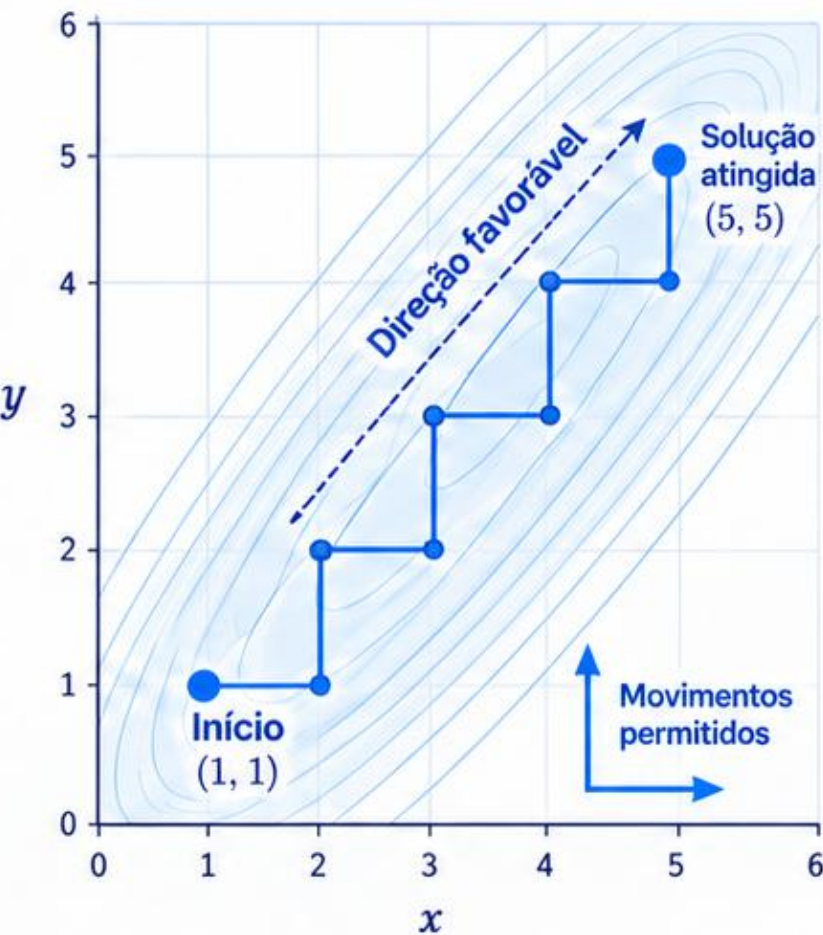
Vizinhança $\mathcal{N}(x)$

Conjunto de soluções geradas a partir de x através de movimentos elementares.

Exemplo de vizinhança ortogonal

$$\mathcal{N}(x, y) = \{(x + 1, y), (x - 1, y), (x, y + 1), (x, y - 1)\}$$

Apenas movimentos horizontais e verticais (passos unitários).



Trajetória ilustrativa com piorias; não é a execução de Hill Climbing estrito

Exemplo: movimentos apenas nos eixos x e y

$$f(x, y) = x + y - 2(y - x)^2$$

Estado inicial: $(1, 1)$, com $f = 2$.

Vizinhos $(2, 1)$ e $(1, 2)$: $f = 1$. Os outros dois vizinhos têm $f = -1$.

Hill Climbing estrito fica bloqueado em $(1, 1)$.

Movimento diagonal para $(2, 2)$: $f = 4$.

Alterar a vizinhança permite uma melhoria direta.

Principais conclusões

- 1 A melhor direção pode não estar disponível num só movimento.
- 2 O progresso pode ser lento ou ficar bloqueado.
- 3 A vizinhança influencia o desempenho.

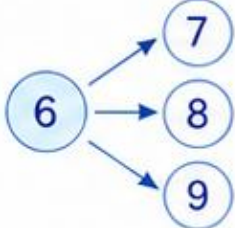
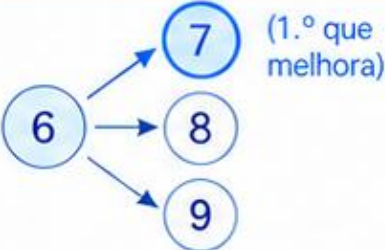
Possíveis alterações à vizinhança

Tipo de vizinhança	Exemplo de movimentos	Comentário
Movimentos diagonais	$(x \pm 1, y \pm 1)$	Permite seguir a crista mais diretamente.
Passos de outro tamanho	$(x \pm s, y), (x, y \pm s)$	Pode acelerar o progresso, mas pode saltar ótimos.
Outra representação	Rotação de coordenadas ou nova descrição	Pode alinhar a crista com os eixos.

Aumentar a vizinhança pode facilitar progresso, mas aumenta o custo de avaliação, pois exige avaliar mais soluções em cada iteração.

Variantes de Hill Climbing

O Hill Climbing é um algoritmo de busca local iterativa que parte de uma solução inicial e, em cada passo, move-se para um vizinho que melhora o valor da função objetivo f . As variantes diferem na forma como selecionam os vizinhos ou na aceitação de movimentos, influenciando o custo computacional e a capacidade de escapar de ótimos locais.

Variante	Regra de seleção / aceitação	Ilustração (vizinhos)	Observações
Melhor melhoria Avalia a vizinhança completa.	Avalia todos os vizinhos $N(s)$ e escolhe o que tem o maior valor de f , desde que $f(v) > f(s)$. $v^* = \arg \max_{v \in N(s), f(v) > f(s)} f(v)$		- Mais avaliações por iteração ($N(s)$). - Maior ganho por passo. - Determinístico com desempate fixo.
Primeira melhoria Pode poupar avaliações; depende da ordem.	Examina os vizinhos um a um e move-se para o primeiro com $f(v) > f(s)$, segundo uma ordem definida.		- Menos avaliações (pode parar cedo). - Depende da ordem de geração dos vizinhos. - Determinístico (para uma dada ordem).
Estocástico Introduz diversidade na escolha.	Gera o conjunto de melhorias $M = \{v \in N(s) : f(v) > f(s)\}$ e escolhe aleatoriamente um vizinho de M .		- Introduz aleatoriedade. - Pode evitar algumas trajetórias desfavoráveis, mas não sai de um ótimo local estrito. - Pode ser combinado com mecanismos de reinício.
Movimentos laterais Ajuda em planaltos; limitar ciclos.	Modificação da aceitação: permite aceitar um vizinho com $f(v) = f(s)$ durante um número limitado de passos L . Aceita se $f(v) > f(s)$ ou $(f(v) = f(s) \text{ e } l < L)$.	 Movimento lateral ($f(v) = f(s)$)	- Não é uma seleção independente de vizinhos. - Útil em planaltos (p.ex., funções com muitos estados de igual valor). - Deve-se limitar o número L de movimentos laterais para evitar ciclos.

Exemplo comum a todas as variantes

Solução atual com $f(s) = 6$ e três vizinhos com melhorias possíveis: $f = 7, 8, 9$.

- Melhor melhoria:** escolhe 9.
- Estocástico:** pode escolher 7, 8 ou 9.
- Primeira melhoria:** escolhe a primeira melhoria encontrada (depende da ordem de exame).
- Movimentos laterais:** só relevam se existir um vizinho com $f(v) = 6$; se apenas existir 7, 8 e 9, comporta-se como uma das variantes acima.

Paisagem da função objetivo (ilustração)



➔ **As variantes mudam a seleção ou a aceitação dos movimentos.** Mantêm a mesma ideia base: melhoria iterativa na vizinhança.

Reinícios aleatórios

Ideia e motivação

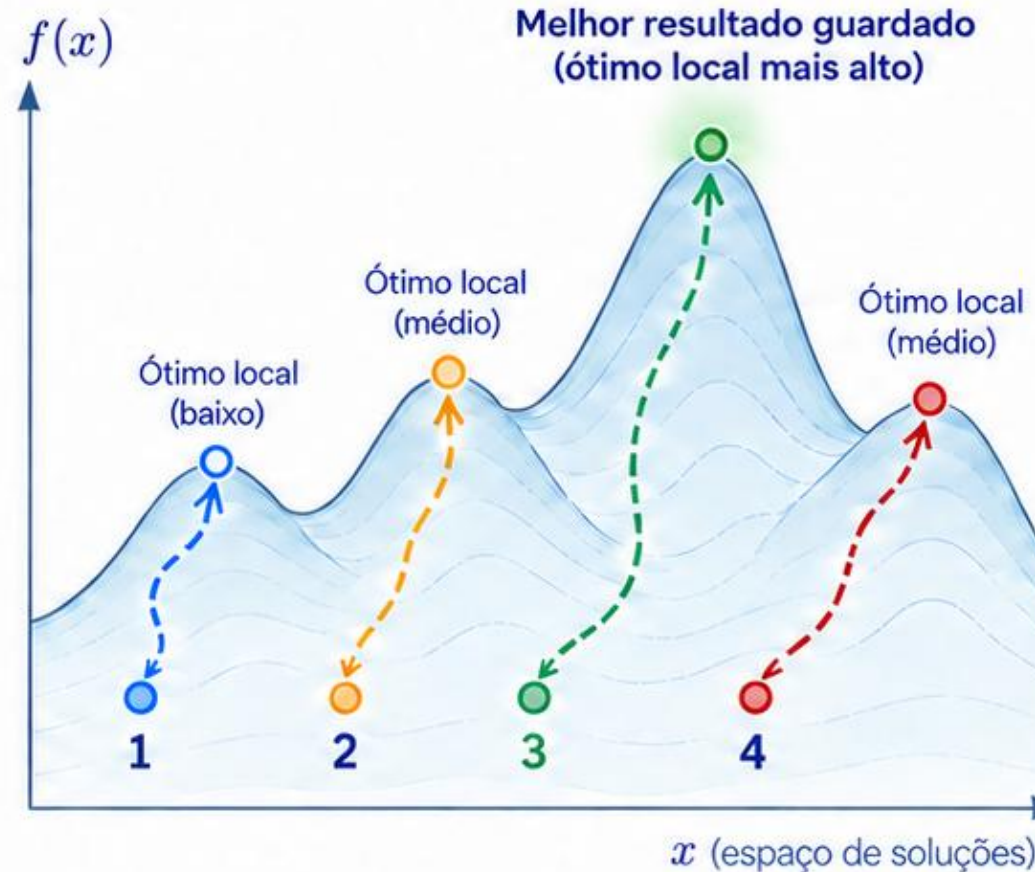
O método de reinícios aleatórios executa várias vezes um algoritmo de otimização (por exemplo, busca local) com soluções iniciais aleatórias, guardando o melhor resultado encontrado. É útil quando o espaço de busca tem vários ótimos locais: diferentes inícios podem conduzir a soluções de melhor qualidade.

Algoritmo (esquema) (maximização)

ReiníciosAleatórios(k, A)

```
1:  $x^* \leftarrow \emptyset, f^* \leftarrow$  menos infinito
2: para  $i = 1, \dots, k$  fazer
3:    $x_0 \leftarrow$  SoluçãoInicialAleatória()
4:    $x \leftarrow A(x_0)$   $\triangleright$  executar algoritmo  $A$ 
5:   se  $f(x) > f^*$  então
6:      $x^* \leftarrow x, f^* \leftarrow f(x)$ 
7:   fim se
8: fim para
9: devolver  $x^*, f^*$ 
```

Várias execuções, diferentes resultados



Probabilidade de sucesso

Se cada execução independente tiver probabilidade p de sucesso (encontrar uma solução satisfatória), então a probabilidade de sucesso em pelo menos uma das k execuções é:

$$P(\text{sucesso em } k \text{ execuções}) = 1 - (1 - p)^k$$

Exemplo:

$$p = 0,2; k = 5 \Rightarrow P = 1 - (1 - 0,2)^5 = 1 - 0,8^5 \\ = 1 - 0,32768 = 0,67232 \approx 67,2\%.$$

Exemplo hipotético; assume independência e p constante.

Considerações práticas

- Dividir o orçamento de computação entre execuções (p. ex., tempo limite ou nº de iterações por execução).
- Conservar o melhor resultado encontrado.
- Mais reinícios aumentam a probabilidade de sucesso, mas também o número total de avaliações.
- Não há garantia finita de encontrar o ótimo global.



Executar

Gerar um início aleatório e executar o algoritmo A .



Guardar o melhor

Se o resultado for melhor, atualizar o melhor guardado.



Novo início aleatório

Gerar uma nova solução inicial aleatória.



Repetir

Repetir até esgotar o orçamento (k execuções).

Mais reinícios: mais oportunidades e mais avaliações. Não há garantia de sucesso global.

Simulated Annealing

Recozimento simulado

1. Ideia e funcionamento

Meta-heurística estocástica: combina uma regra de aceitação com um plano de temperatura para explorar o espaço de soluções e evitar ficar preso em ótimos locais.

A partir de uma solução atual s , gera-se uma solução vizinha s' , com variação de custo $\Delta = f(s') - f(s)$ (problema de minimização).

Regra de aceitação (Metropolis)	$P(\text{aceitar } s') = \begin{cases} 1, & \text{se } \Delta \leq 0 \\ \exp(-\Delta/T), & \text{se } \Delta > 0 \end{cases}$
---------------------------------	---

$\Delta = f(s') - f(s)$: variação do custo ($\Delta > 0$ é uma pioria).
 T : temperatura ($T > 0$).
Se $\Delta \leq 0$ a solução é sempre aceite. Se $\Delta > 0$ é aceite com probabilidade $\exp(-\Delta/T)$.
À medida que T diminui, a aceitação de piorias torna-se menos provável.

3. Exemplo numérico de aceitação

Solução atual com custo $f(s) = 100$. Vizinha com $f(s') = 110$ (pioria, $\Delta = 10$).
Calcule a probabilidade de aceitar para diferentes temperaturas:

Temperatura T	$\Delta = 10$	$P(\text{aceitar}) = \exp(-\Delta/T)$	Interpretação
100	10	$\exp(-10/100) = 0,905$	Porias muito prováveis
10	10	$\exp(-10/10) = 0,368$	Porias ainda possíveis
1	10	$\exp(-10/1) = 0,000045$	Porias muito improváveis

2. Exploração do espaço de soluções



Melhoria imediata

- Exploração restrita à região atual.
- Pode ficar preso em ótimos locais.
- Não ultrapassa barreiras de custo.

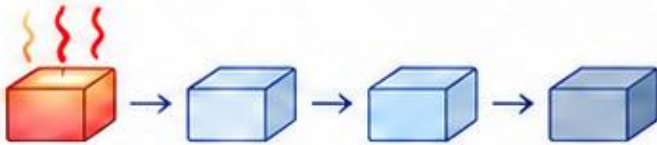
Aceitação de piorias

- Oportunidade de ultrapassar barreiras.
- Maior exploração do espaço de soluções.
- Pode alcançar soluções de melhor qualidade.

4. Plano de temperatura (algoritmo)

- 1 Inicializar $s, T = T_0$
- 2 Repetir até critério de paragem:
 - Gerar vizinha s'
 - Calcular $\Delta = f(s') - f(s)$
 - Aceitar s' com a regra de Metropolis
 - Atualizar T (ex.: $T \leftarrow \alpha T$, com $0 < \alpha < 1$)
- 3 Devolver a melhor solução encontrada

5. Inspiração física



Inspiração: arrefecimento de materiais

Não aceita todas as piorias e não garante escapar em cada execução.

Exemplos de aplicação: 8 rainhas, horários, caixeiro-viajante.

A aceitação depende da temperatura e da dimensão da pioria.

Temperatura e exploração

1. Conceito e papel da temperatura

No algoritmo de arrefecimento simulado (*simulated annealing*), a temperatura T controla a probabilidade de aceitar soluções piores, permitindo escapar de ótimos locais. No início, T é alta e o algoritmo explora amplamente o espaço de soluções; à medida que T diminui, torna-se mais seletivo e foca a exploração em torno das melhores soluções encontradas.

2. Esquema de arrefecimento

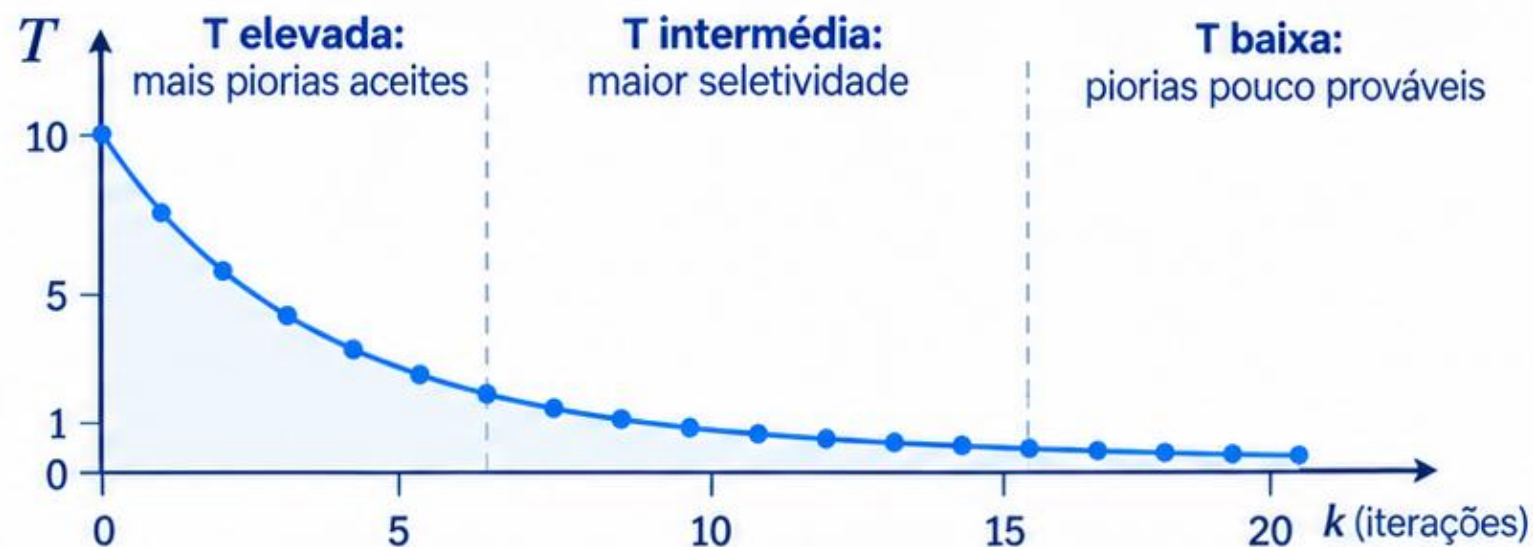
Exemplo: arrefecimento geométrico

$$T_{k+1} = \alpha T_k, \quad 0 < \alpha < 1$$

$$T_0 = 10; \quad \alpha = 0,8: \quad 10 \rightarrow 8 \rightarrow 6,4 \rightarrow 5,12$$

- α próximo de 1: arrefecimento mais lento.
- **Temperatura inicial** deve ser compatível com a escala dos custos.
- **Definir quantas** propostas são testadas por temperatura.
- **Arrefecimento lento** dá mais exploração, mas exige maior orçamento.

3. Efeito da temperatura na exploração



Arrefecer demasiado depressa pode limitar a exploração.

Probabilidade de aceitar uma pioria

Minimização de custo

Em **problemas de minimização**, um algoritmo de procura local como o recozimento simulado (Simulated Annealing) pode aceitar soluções piores com uma certa probabilidade, permitindo explorar melhor o espaço de soluções e evitar ótimos locais.

1. Regra de aceitação

Considere uma solução atual e um vizinho. Seja

$$\Delta = C(\text{vizinho}) - C(\text{atual})$$

- $\Delta \leq 0$: **aceitar** (a solução é melhor ou igual).
- $\Delta > 0$: **aceitar** com probabilidade

$$P = \exp(-\Delta/T), \quad T > 0$$

onde T é a temperatura de controlo.

2. Amostragem

Sortear um número aleatório $u \sim U[0, 1)$ e aceitar a pioria se

$$u < P$$

3. Exemplo de cálculo

Considere $C(\text{atual}) = 4$ e $C(\text{vizinho}) = 6$.
Então $\Delta = 2$ (pioria). A probabilidade de aceitação é

$$P = \exp(-2/T)$$

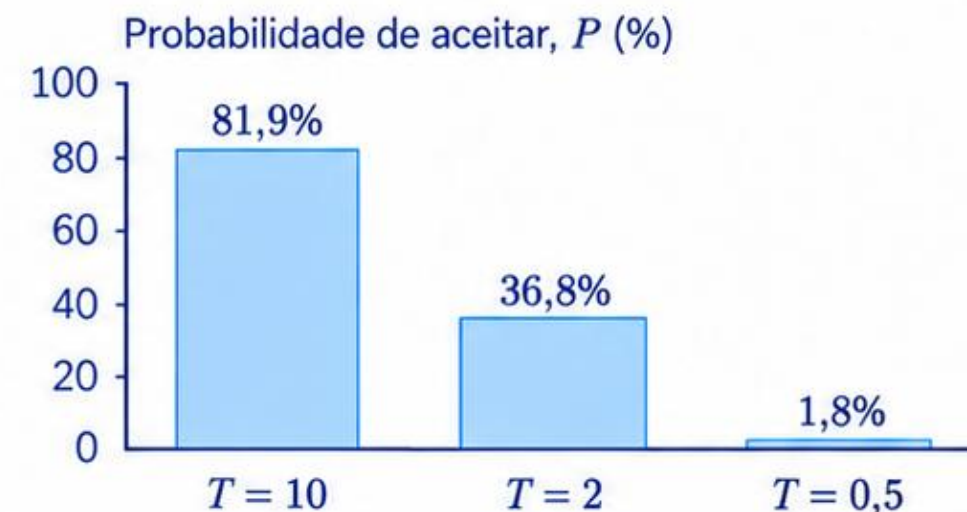
Para diferentes temperaturas:

T	$P = \exp(-2/T)$	Probabilidade de aceitar
10	$\exp(-0,2) = 0,8187$	81,9%
2	$\exp(-1) = 0,3679$	36,8%
0,5	$\exp(-4) = 0,0183$	1,8%

Exemplo ($T = 2$): $P \approx 0,368$.

- Se $u = 0,20 < 0,368 \rightarrow$ aceita a pioria.
- Se $u = 0,70 > 0,368 \rightarrow$ rejeita a pioria.

4. Comportamento da probabilidade



5. Observações importantes

- Para $\Delta = 0$ (mesmo custo), $P = \exp(0) = 1 \rightarrow$ aceitar.
- Δ maior $\Rightarrow$ menor P .
- T maior $\Rightarrow$ maior P .
- Quando T diminui, a aceitação de piorias torna-se menos provável, tornando a procura mais seletiva.

A mesma pioria torna-se menos provável quando T diminui.

Simulated Annealing: algoritmo

Meta-heurística estocástica para problemas de otimização combinatória e contínua

O Simulated Annealing (SA) é uma meta-heurística inspirada no processo físico de recozimento, que permite aceitar soluções piores com uma certa probabilidade, evitando mínimos locais e explorando o espaço de soluções.

Algoritmo (pseudocódigo)

```
1  atual ← inicializar(); melhor ← atual
2  T ← T0
3  enquanto T > Tmin e houver orçamento:
4      v ← vizinho aleatório de atual
5      Δ ← C(v) – C(atual)
6      se Δ ≤ 0 ou U(0,1) < exp(–Δ/T):
7          atual ← v
8      se C(atual) < C(melhor): melhor ← atual
9      T ← αT
10 devolver melhor
```

$U(0, 1)$ é uma variável aleatória uniforme em $[0, 1)$; $T > 0$.

Exemplo numérico (uma iteração)

Estado atual	Vizinho v	$C(\text{atual})$	$C(v)$	Δ	T	$p = \exp(-\Delta/T)$	u	Decisão
x	x'	100	110	10	5	$\exp(-10/5) = 0,135$	0,08	Aceitar (piora)

Como $u = 0,08 < 0,135$, a solução pior é aceite, permitindo escapar de mínimos locais.

Critério de aceitação

$$p = \begin{cases} 1, & \text{se } \Delta \leq 0 \\ \exp(-\Delta/T), & \text{se } \Delta > 0 \end{cases}$$

- $\Delta = C(v) - C(\text{atual})$ é a variação de custo.
- Se $\Delta \leq 0$, a solução é sempre aceite.
- Se $\Delta > 0$, a solução pior é aceite com probabilidade $\exp(-\Delta/T)$.
- Rejeitar mantém o estado atual.

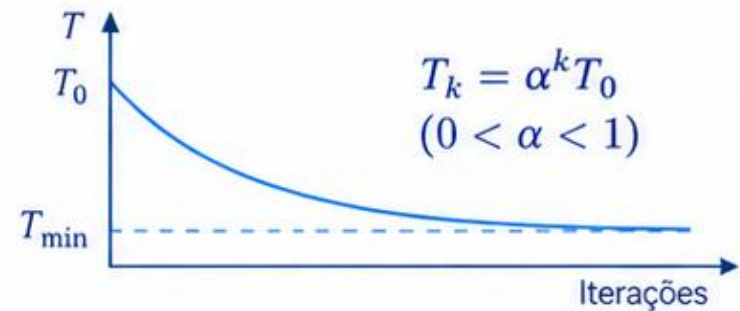
Parâmetros

T_0	temperatura inicial
$T_{\min} > 0$	limite inferior da temperatura
$0 < \alpha < 1$	fator de arrefecimento
C	custo a minimizar

Registos de soluções

Solução atual	pode piorar (exploração)
Melhor encontrada	conserva o menor custo (observada até agora)

Esquema do arrefecimento



Paragem: orçamento de avaliações (número máximo de iterações/avaliações da função de custo) ou temperatura mínima ($T \leq T_{\min}$).

Simulated Annealing nas 8 rainhas

Exemplo de um passo do algoritmo: gerar um vizinho, calcular Δ e decidir pela regra de aceitação

1 Problema e formulação

Colocar 8 rainhas num tabuleiro 8×8 de modo a que nenhuma se ataque. Representamos um estado por um vetor de 8 posições, onde a i -ésima entrada indica a linha da rainha na coluna i .

Função de avaliação

$h(s)$ = número de pares de rainhas em conflito (mesma linha ou mesma diagonal).

Objetivo: minimizar $h(s)$ ($h(s) = 0$ é uma solução).

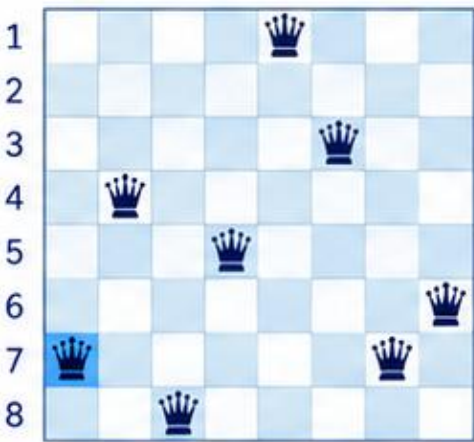
Simulated Annealing (minimização)

- 1 A partir do estado atual s , gerar um vizinho s' .
- 2 Calcular $\Delta = h(s') - h(s)$.
- 3 Se $\Delta \leq 0$, aceitar s' .
- 4 Se $\Delta > 0$, aceitar s' com probabilidade $P = \exp(-\Delta/T)$.
- 5 Atualizar a temperatura T segundo um esquema de arrefecimento (ex.: $T_{k+1} = \alpha T_k$, $0 < \alpha < 1$).

2 Exemplo de um passo

Estado atual e vizinho candidato (o movimento sorteado não tem de ser o melhor vizinho).

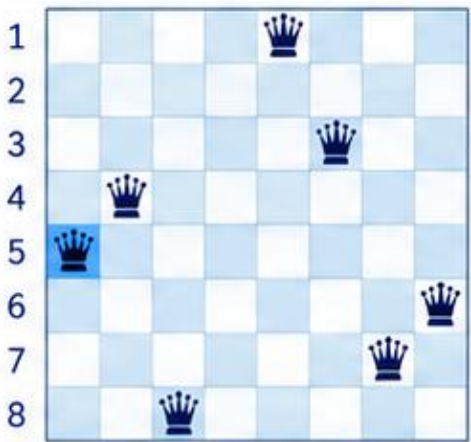
Atual: $h = 4$



Vetor (linhas 1..8):

7	4	8	5	1	3	7	6
---	---	---	---	---	---	---	---

Candidato: $h = 6$



Vetor (linhas 1..8):

5	4	8	5	1	3	7	6
---	---	---	---	---	---	---	---

Coluna 1: linha 7 $\rightarrow$ linha 5

$\Delta = 2$; $T = 2$; $P = \exp(-2/2) \approx 0,368$

$u = 0,20$: aceitar ($0,20 < 0,368$)

$u = 0,70$: rejeitar ($0,70 > 0,368$)

3 Atualização do estado

Mantemos o melhor estado encontrado separadamente do estado atual.

Situação	Estado atual $h(s)$	Melhor estado $h(s^*)$
Antes	$h = 4$	$h = 4$
Se aceitar	$h = 6$	$h = 4$
Se rejeitar	$h = 4$	$h = 4$

Regra de aceitação

$$\text{Aceitar } s' = \begin{cases} 1, & \text{se } \Delta \leq 0 \\ \exp(-\Delta/T), & \text{se } \Delta > 0 \end{cases}$$

Uma pioria pode ser aceite com probabilidade P , o que permite escapar de ótimos locais. A melhor solução encontrada é sempre guardada.

Observações

- O movimento sorteado não tem de ser o melhor vizinho.
- Este passo ilustra a aceitação, não o resultado final da procura.
- Com o arrefecimento, T diminui e a aceitação de piorias torna-se menos provável.

Uma pioria pode ser aceite; a melhor solução permanece guardada.

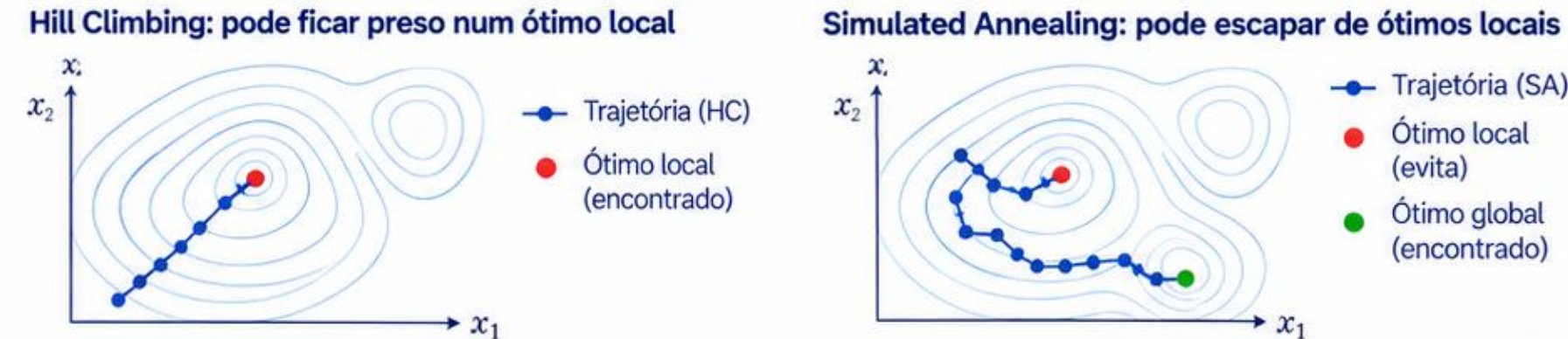
Hill Climbing vs. Simulated Annealing

Comparação com a versão de melhor melhoria

1. Comparação dos algoritmos

Aspeto	Hill Climbing básico	Simulated Annealing
Vizinho	Melhor vizinho	Aleatório
Piorias	Rejeita	Pode aceitar
Ótimos locais	Pode bloquear	Pode escapar
Parâmetros	Estado e vizinhança	Também temperatura e arrefecimento
Ótimo global em tempo limitado	Sem garantia	Sem garantia
Avaliações por proposta	Até todos os vizinhos	Um vizinho proposto
Reprodutibilidade	Depende do início e dos empates	Depende também da semente

3. Comportamento na paisagem da função objetivo



Notas importantes

- Comparar pelo número de avaliações da função, não apenas por iterações.
- Temperatura baixa não significa escolher o melhor vizinho.
- Convergência teórica do recozimento exige condições e arrefecimento apropriados; não é garantia numa execução finita.

2. Ideia e algoritmos

Hill Climbing (HC) é uma busca local gulosa: em cada iteração move-se para o melhor vizinho que melhore o valor da função objetivo $f(x)$.

Hill Climbing (melhor melhoria)

1. Começar em x (inicial).
2. Avaliar $f(x')$ para todos os vizinhos $x' \in \mathcal{N}(x)$.
3. Se existir x' com $f(x') < f(x)$, então $x \leftarrow \arg \min_{x' \in \mathcal{N}(x)} f(x')$.
4. Caso contrário, parar (ótimo local).

Simulated Annealing (SA) permite aceitar piorias com probabilidade

$$P(\text{aceitar}) = \exp\left(-\frac{\Delta f}{T}\right), \quad \Delta f = f(x') - f(x).$$

A temperatura T diminui segundo um esquema de arrefecimento.

Simulated Annealing

1. Começar em x , com temperatura inicial T_0 .
2. Escolher um vizinho aleatório $x' \in \mathcal{N}(x)$ e calcular $\Delta f = f(x') - f(x)$.
3. Se $\Delta f \leq 0$, aceitar x' ; caso contrário, aceitar com probabilidade $P = \exp\left(-\frac{\Delta f}{T}\right)$.
4. Atualizar a temperatura: $T \leftarrow \alpha T$ ($0 < \alpha < 1$).
5. Repetir até ao critério de paragem.

Exercício final: experimentar e comparar

Experiência: 8 rainhas

Colocar 8 rainhas num tabuleiro 8×8 sem que se ataquem. Usar busca local com diferentes métodos.

Representação e função de custo

- **Estado:** vetor $x = (x_1, \dots, x_8)$, onde $x_i \in \{1, \dots, 8\}$ indica a linha da rainha na coluna i .
- **Vizinhança:** todos os estados obtidos ao mover uma rainha para outra linha na mesma coluna (máximo $8 \times 7 = 56$ vizinhos).
- **Função de custo $h(x)$:** número de pares de rainhas em conflito (mesma linha ou diagonal).

$$h(x) = \sum_{1 \leq i < j \leq 8} [x_i = x_j] + [|x_i - x_j| = |i - j|]$$

Métodos a comparar

- 1 **Hill Climbing (HC)**
 - Em cada passo, escolher o vizinho com menor h .
 - Parar quando não houver melhora (mínimo local).
- 2 **Hill Climbing com reinícios (HC+R)**
 - Executar o HC a partir de vários estados iniciais aleatórios.
 - Todos os reinícios partilham o orçamento da execução.
- 3 **Simulated Annealing (SA)**
 - Aceitar sempre melhoras e, por vezes, piores soluções.
 - Probabilidade de aceitação de um pior vizinho ($\Delta h > 0$):
$$P(\text{aceitar}) = e^{-\Delta h / T}$$
 - Temperatura a decrescer segundo um esquema $T(k)$ (escolher e justificar, por exemplo, $T(k) = T_0 \cdot \alpha^k$).

Comparação justa

Usar o mesmo problema, a mesma função de custo e a mesma vizinhança, garantindo uma comparação justa.

- 1 **Mesma função de custo e vizinhança**

Todos os métodos usam a mesma definição de estado, vizinhança e $h(x)$.
- 2 **Mesmo orçamento de avaliações**

Cada execução pode avaliar no máximo 10 000 estados (avaliações de h), independentemente do método.
- 3 **Vários estados iniciais e sementes**

Realizar 30 execuções independentes por método, com diferentes estados iniciais aleatórios e sementes.

Proposta de protocolo (não são resultados)

1. Implementar os três métodos com a mesma função de custo $h(x)$.
2. Executar 30 execuções independentes por método.
3. Em cada execução, permitir até 10 000 avaliações de h .
4. Parar uma execução quando $h = 0$ (solução) ou ao esgotar o orçamento.
5. No HC+R, todos os reinícios de uma execução partilham o limite total de 10 000 avaliações.
6. Registrar para cada execução: semente aleatória, parâmetros do método (ex.: T_0 e α no SA) e número de avaliações usadas.



Não são apresentados resultados nesta ficha.

O objetivo é que implementes, executes e analyses os resultados seguindo este protocolo, garantindo a reprodutibilidade.

Medir

Avaliar e comparar o desempenho de forma rigorosa.

- 1 **Sucesso: $h = 0$**

Uma execução tem sucesso se encontrar uma solução com $h = 0$.
- 2 **Custo final médio e melhor custo**

Calcular a média e o melhor valor de h obtido nas execuções (incluindo as que não têm sucesso).
- 3 **Avaliações e variabilidade**

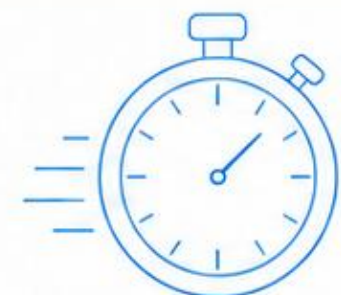
Registrar o número de avaliações de h usadas e medir a variabilidade entre execuções (ex.: desvio padrão).

Métricas e apresentação

- Taxa de sucesso:

$$\text{Taxa de sucesso} = \frac{\text{execuções com } h = 0}{\text{total de execuções}}$$

- Apresentar a média e a dispersão (ex.: desvio padrão) do custo final e do número de avaliações.
- Justificar os parâmetros escolhidos (ex.: T_0 , α no SA) com base no comportamento observado.
- Discutir vantagens, desvantagens e robustez de cada método.
- Registrar todas as sementes e parâmetros para permitir a reprodutibilidade dos resultados.



Até
10 000
avaliações
por execução

Que método escolherias? Justifica com os resultados.